

Université de Nouakchott
Faculté des Sciences et Techniques



Rapport du module Projet :

CampusConnect

Plateforme de gestion de la vie associative et estudiantine

Saleck Ahmed Baya (C25362)

Moussa Mohamed Erebih (C21397)

Filière : SI (Parcours Systèmes d'Information)

Table des matières

Résumé.....	4
Première partie : Introduction et Contexte	5
1 Introduction et Contexte.....	5
2 Objectifs du Projet.....	5
Deuxième partie : II Architecture Technique	6
3 Stack Technique	6
4 Structure du Projet et Base de Données.....	6
4.1 Organisation des dossiers	6
4.2 Schéma de la Base de Données	7
5 La Conception UML.....	7
5.1 Diagramme de cas d'utilisation	8
5.2 Diagramme de séquence (authentification).....	9
5.3 Diagramme de séquence (création d'une association)	10
5.4 Diagramme de séquence (création d'un événement).....	11
5.5 Diagramme de classe	12
Troisième partie : III Fonctionnalités et Modèle Économique	13
6 Fonctionnalités Détaillées.....	13
6.1 Gestion des Identités et Accès (IAM / RBAC)	13
6.2 Gestion des Événements.....	13
6.3 Gestion des Associations et ONG	13
6.4 Panel Administrateur	14
7 Modèle Économique	14
7.1 Sources de Revenus	14
7.2 Fonctionnement du Système de Paiement (MVP).....	14
Quatrième partie : IV Méthodologie et Qualité	15
8 Méthodologie Scrum Adaptée et Suivi des Sprints	15
8.1 Adaptations de la Méthodologie Scrum	15
8.2 Suivi Détaillé des Sprints.....	15
8.3 Synthèse des Sprints.....	18
9 Sécurité et Qualité du Code	19
9.1 Authentification et Autorisation	19

9.2 Sécurité du Serveur	19
10 Tests et Audit	20
10.1 Tests Fonctionnels Validés	20
10.2 Points d'Attention Identifiés	20
Cinquième partie : V Bilan et Conclusion	21
11 Bilan et Perspectives	21
11.1 Ce qui a été livré	21
11.2 Perspectives v2.0	21
Conclusion.....	22

Résumé

CampusConnect est une plateforme web full-stack développée pour moderniser et centraliser la gestion de la vie associative étudiante à la Faculté des Sciences et Techniques. Elle offre un ensemble d'outils performants pour les étudiants, les associations (clubs et ONG), et l'administration universitaire : gestion des événements avec algorithme anti-conflit de salles, automatisation des adhésions et de la billetterie, et suivi de trésorerie en temps réel.

Conçue avec une architecture robuste (React / Node.js / MySQL) et sécurisée par un modèle RBAC, la solution intègre un modèle économique Freemium viable, soutenu par des abonnements Premium, des commissions automatisées et des partenariats corporatifs. Ce rapport détaille l'architecture technique, les fonctionnalités clés, le modèle d'affaires, et la méthodologie de développement Scrum adaptée et mise en œuvre pour mener à bien ce projet.

Mots-Clés : Vie associative, Gestion d'événements, SaaS Freemium, Architecture Client-Serveur, React.js, Node.js, MySQL, RBAC, Scrum, Automatisation.

Première partie : Introduction et Contexte

1 Introduction et Contexte

CampusConnect est une solution logicielle Full-stack développée dans le cadre d'un projet universitaire à la Faculté des Sciences et Techniques (FST). Elle répond à un besoin concret identifié au sein de l'université : le manque de visibilité et d'organisation des associations étudiantes.

Problèmes identifiés

- Aucune centralisation des informations sur les associations et leurs activités.
- Gestion des inscriptions aux événements dispersée (papier, messagerie informelle).
- Pas de suivi financier pour les responsables d'association.
- Aucun système de réservation de salles empêchant les conflits de planning.
- Visibilité quasi nulle des ONG et associations à vocation sociale.

CampusConnect centralise ces flux en une seule plateforme web accessible à tous les acteurs : étudiants, responsables d'associations, ONG, et administrateurs.

2 Objectifs du Projet

Les objectifs principaux de ce projet sont les suivants :

- **Centraliser** : Regrouper toutes les associations, événements, ressources et actualités sur une seule plateforme.
- **Automatiser** : Éliminer les tâches manuelles : inscriptions, paiements, notifications, cotisations.
- **Sécuriser** : Mettre en place un système RBAC (4 rôles) avec authentification JWT et chiffrement bcrypt.
- **Monétiser** : Créer une plateforme viable économiquement via le Freemium, les commissions et les sponsors.
- **Piloter** : Offrir des tableaux de bord temps réel aux responsables et à l'administrateur.

Deuxième partie : II Architecture Technique

3 Stack Technique

CampusConnect repose sur une architecture Client-Serveur découplée, permettant une évolution indépendante du frontend et du backend.

TABLE 3.1 – Stack Technique

Couche	Technologie	Rôle / Justification
Frontend	React 18 + Vite	Rendu rapide, composants réutilisables, hot-reload
Styling	Vanilla CSS (Design System)	Maîtrise totale de l'UI/UX, identité visuelle Premium
Backend	Node.js + Express	Asynchrone, performance, architecture MVC REST
Base de données	MySQL + mysql2	Robustesse transactionnelle, intégrité des données
Authentification	JWT + bcrypt Round 10	Tokens stateless, sécurisation des mots de passe
Emails	Nodemailer + cron	Notifications automatiques, rappels planifiés
Documentation API	Swagger / OpenAPI 3.0	Documentation interactive testable sur /api-docs
Uploads	Multer	Gestion des logos et preuves de paiement

4 Structure du Projet et Base de Données

4.1 Organisation des dossiers

- backend/controllers/ : Logique métier (Auth, Events, Associations, Donations, Sponsors)
- backend/middlewares/ : Sécurité : JWT, RBAC (requireRole), gestion des uploads (Multer)
- backend/routes/ : Définition des endpoints REST
- backend/models/ : Abstraction SQL avec requêtes préparées
- backend/utils/ : Services email (Nodemailer), tâches planifiées (cron)
- frontend/src/pages/ : Vues : Home, Events, AssociationDetails, AdminPanel, ResponsablePanel
- frontend/src/components/ : Composants réutilisables : DonateModal, SponsorModal, Navbar

- frontend/src/context/ : AuthContext : état global de l'utilisateur connecté
- sql/ : Schéma de la base de données

4.2 Schéma de la Base de Données

TABLE 4.1 – Tables principales de la Base de Données

Table	Colonnes principales
users	id, name, email, password_hash, role, is_active, created_at
associations	id, name, description, logo_url, plan, is_validated, premium_until, commission_percent, type
association_members	id, user_id, association_id, status, cotisation_amount, cotisation_status
events	id, title, start_date, end_date, location, capacity, price_guest, price_member, association_id
registrations	id, user_id, event_id, price_paid, payment_status, payment_proof_url
commissions	id, association_id, source_type, amount, created_at
sponsors	id, name, logo_url, website, is_active, start_date, end_date, amount_paid
platform_reviews	id, user_id, rating, comment, created_at

5 La Conception UML

Cette section présente les diagrammes UML élaborés lors de la phase de conception de CampusConnect. Ils couvrent les cas d'utilisation des différents acteurs, les interactions dynamiques entre composants, ainsi que la structure statique des données du système.

5.1 Diagramme de cas d'utilisation

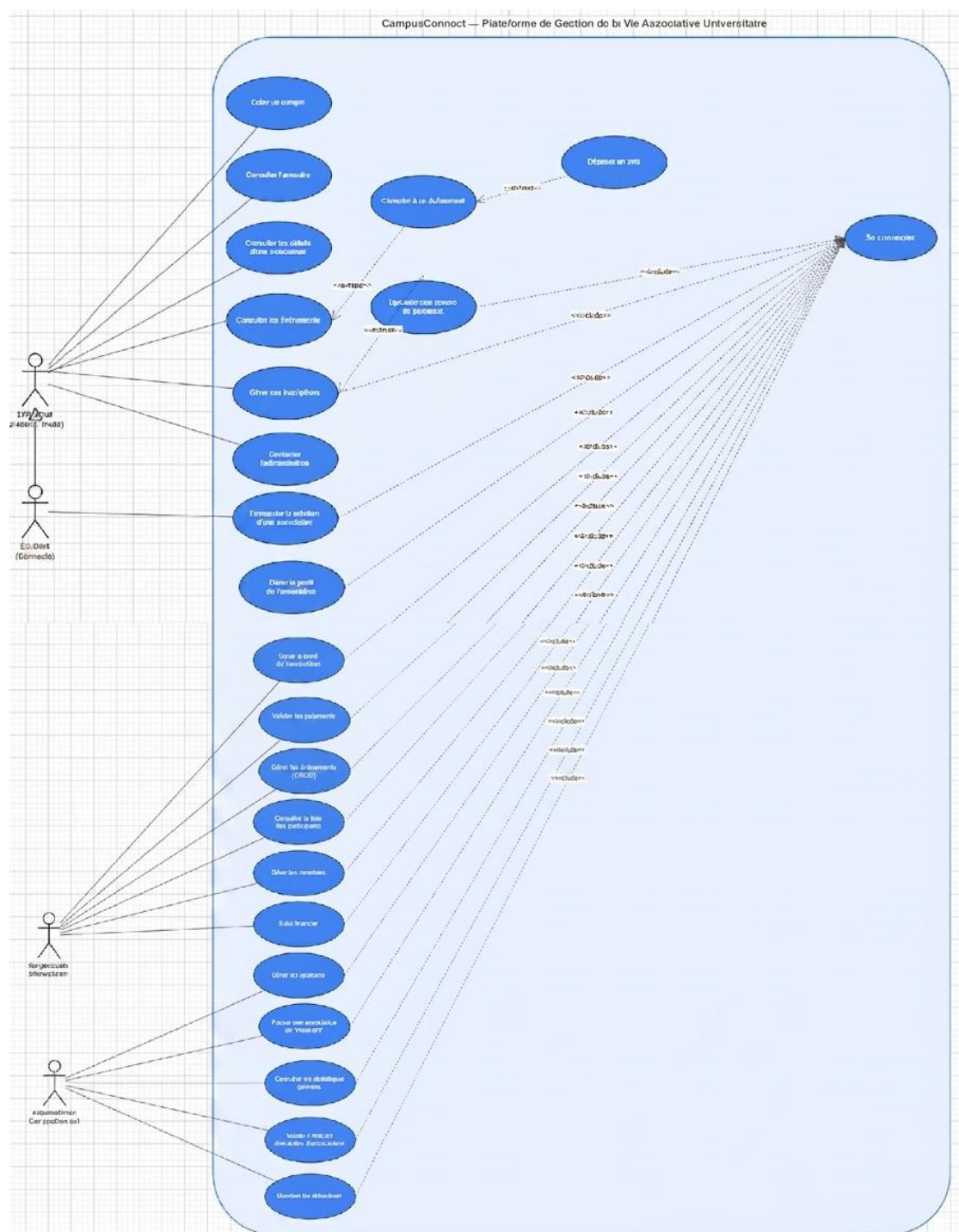


Diagramme de cas d'utilisation : CampusConnect

Figure 5.1 – Diagramme de cas d'utilisation de la plateforme CampusConnect. Il recense l'ensemble des fonctionnalités accessibles aux quatre acteurs du système : l'**Invité** (consultation de l'annuaire et des événements), l'**Étudiant** (inscription, adhésion, gestion du profil), le **Responsable d'association** (création d'événements, validation des paiements, gestion des membres et suivi de trésorerie) et l'**Administrateur** (validation des associations, gestion des sponsors et modération des utilisateurs). Le cas d'utilisation central *Se connecter* est inclus par la majorité des actions nécessitant une authentification.

5.2 Diagramme de séquence (authentification)

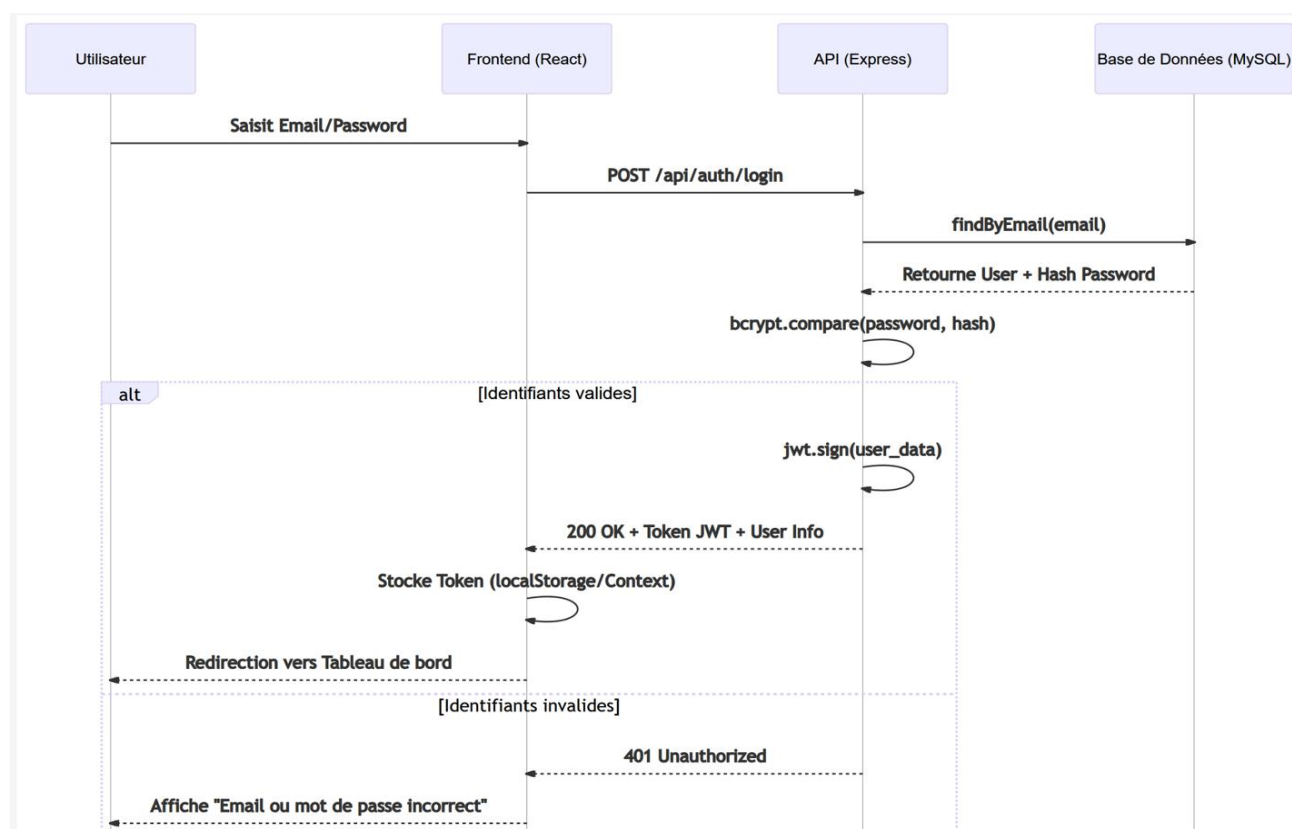


Diagramme de séquence : Authentification JWT

Figure 5.2 – Diagramme de séquence du flux d'authentification. L'utilisateur soumet ses identifiants via le frontend React, qui émet une requête POST /api/auth/login vers l'API Express. Celle-ci interroge la base MySQL pour récupérer l'enregistrement de l'utilisateur, compare le mot de passe au hash bcrypt stocké, puis génère un token JWT signé en cas de succès (200 OK + Token JWT). Le token est ensuite stocké côté client (localStorage/Context) et l'utilisateur est redirigé vers son tableau de bord. En cas d'identifiants invalides, l'API retourne 401 Unauthorized et le frontend affiche un message d'erreur.

5.3 Diagramme de séquence (création d'une association)

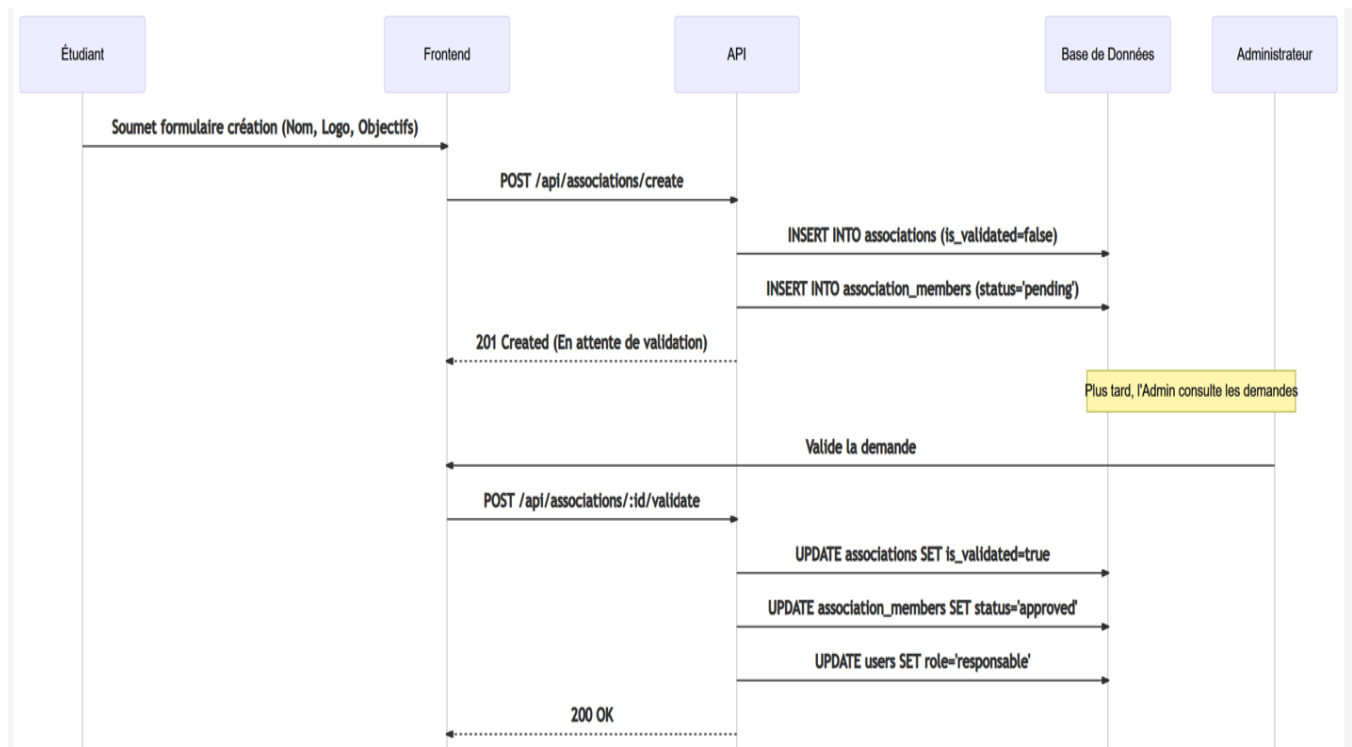


Diagramme de séquence : Création et validation d'une association

Figure 5.3 – Diagramme de séquence du workflow de création d'une association. L'étudiant soumet un formulaire (Nom, Logo, Objectifs) qui déclenche un appel POST /api/associations/create. L'API insère l'association avec le statut is_validated=false et enregistre le demandeur comme membre avec le statut

pending. Dans un second temps, l'administrateur consulte les demandes en attente et valide l'association via `POST /api/associations/:id/validate`, ce qui déclenche en cascade la mise à jour du statut de l'association (`is_validated=true`), du membre (`approved`) et du rôle de l'utilisateur (`responsable`).

5.4 Diagramme de séquence (création d'un événement)

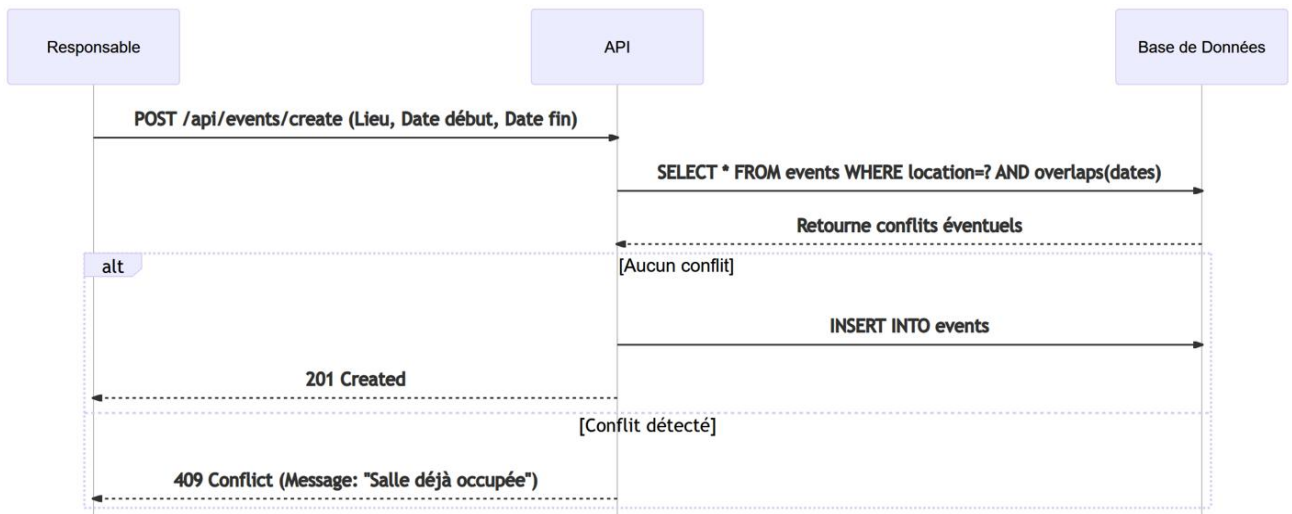


Diagramme de séquence : Création d'un événement avec anti-conflit

Figure 5.4 – Diagramme de séquence de la création d'un événement avec l'algorithme anti-conflit de salles. Le responsable envoie une requête `POST /api/events/create` avec le lieu et les dates de l'événement. L'API exécute une requête SQL de vérification des chevauchements temporels sur la même salle (`SELECT * FROM events WHERE location=? AND overlaps(dates)`). Si aucun conflit n'est détecté, l'événement est inséré en base et une réponse `201 Created` est retournée. En cas de chevauchement, l'API retourne `409 Conflict` avec le message « Salle déjà occupée », sans créer l'événement.

5.5 Diagramme de classe

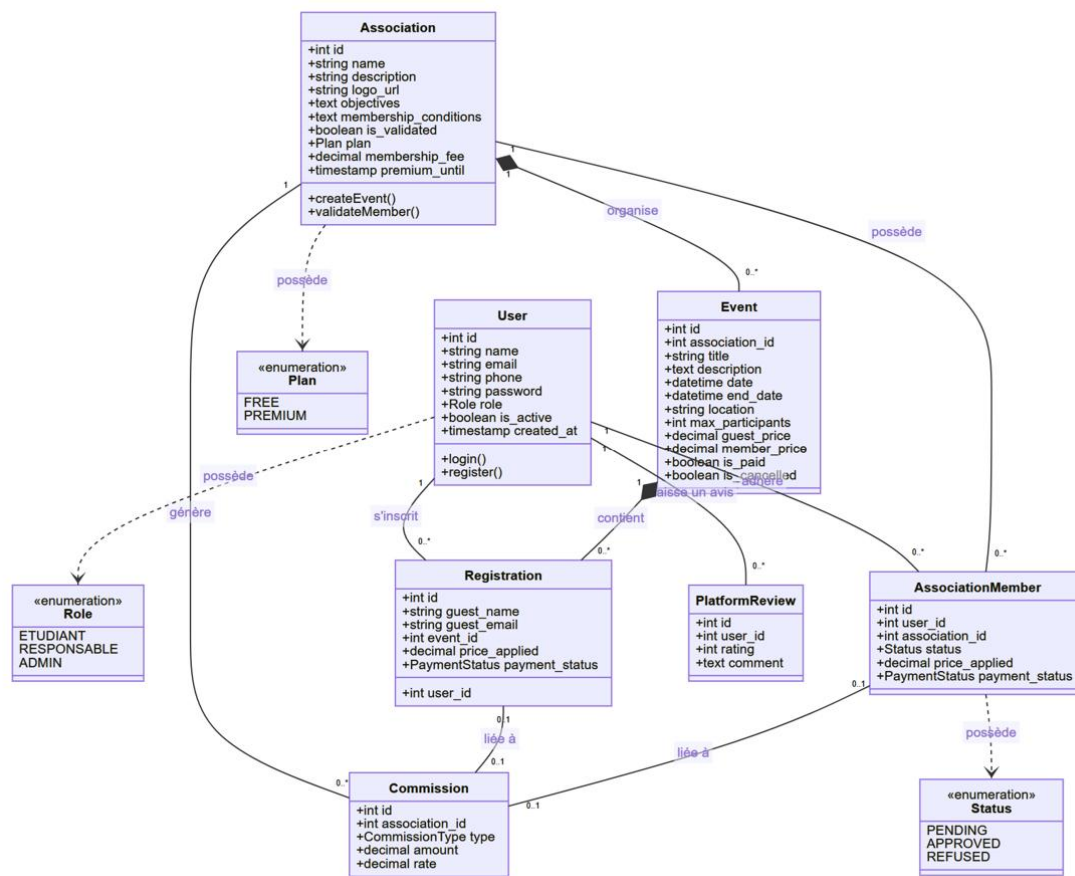


Diagramme de classe : CampusConnect

Figure 5.5 – Diagramme de classe du domaine métier de CampusConnect. Il illustre les principales entités et leurs relations : une **Association** organise des **Events** et possède des **AssociationMembers** ; un **User** s'inscrit à des événements via une **Registration** et laisse des avis via **PlatformReview** ; les transactions financières sont tracées dans la classe **Commission**. Les énumérations **Role** (ETUDIANT, RESPONSABLE, ADMIN), **Plan** (FREE, PREMIUM), **Status** (PENDING, APPROVED, REFUSED) et **PaymentStatus** formalisent les contraintes métier du système RBAC et du modèle Freemium.

Troisième partie : III Fonctionnalités et Modèle Économique

6 Fonctionnalités Détaillées

6.1 Gestion des Identités et Accès (IAM / RBAC)

La plateforme implémente un système RBAC à 4 niveaux : Invité, Étudiant, Responsable et Administrateur. Chaque rôle dispose d'accès spécifiques contrôlés par le middleware `requireRole` côté backend.

- Inscription multi-rôle et activation par email (double opt-in via Nodemailer).
- Authentification JWT (24h) avec refresh implicite et reset mot de passe par email.
- Suspension de compte par l'administrateur sans suppression (champ `is_active`).

6.2 Gestion des Événements

- **Algorithme anti-conflit** (`checkConflict`) : vérification SQL des chevauchements temporels sur une même salle (retourne HTTP 409 Conflict).
- **Tarification dynamique** : prix différencié Invité / Membre, remise 20% pour les 10 premiers inscrits.
- **Cycle de vie complet** : Création → Publication → Inscription → Validation paiement → Archivage automatique.
- **Notifications automatiques** : confirmation d'inscription, rappels J-1 et J-2 via cron.

6.3 Gestion des Associations et ONG

- Annuaire public des associations validées avec fiche détaillée.
- Workflow de création : demande → statut pending → validation admin → promotion responsable.
- Gestion des membres : adhésion, révocation, cotisation avec commission automatique (5%).
- Module de dons pour les ONG avec commission plateforme de 10%.
- Trésorerie temps réel : journal de transactions avec statut de paiement.

6.4 Panel Administrateur

- Validation des associations (clubs et ONG) et des demandes de rôle responsable.
- Gestion des sponsors corporatifs (CRUD complet avec logo, dates, montant).
- Onglet Abonnements : activation et renouvellement Premium des associations.
- Statistiques globales et modération des utilisateurs (suspension/réactivation).

7 Modèle Économique

CampusConnect adopte un modèle Freemium B2B2C, ciblant les associations universitaires comme clients principaux et les étudiants comme utilisateurs finaux.

7.1 Sources de Revenus

TABLE 7.1 – Sources de Revenus (Modèle Freemium)

Source	Montant	Mécanisme
Abonnement Premium	500 MRU/mois	Validation manuelle admin → activation 30 jours → cron de révocation
Commission Billetterie	10% par billet	Calculée à la validation du paiement par le responsable
Commission Cotisation	5% de la cotisation	Calculée lors de la validation de l'adhésion d'un membre
Sponsor Corporatif	5 000 MRU/mois	Logo landing page, espace partenaires, modal contact
Visibilité Premium	Inclus Premium	Événements à la une, carousel homepage, badge exclusif

7.2 Fonctionnement du Système de Paiement (MVP)

En l'absence d'intégration bancaire (MVP), le système fonctionne comme un livre de comptes:

1. L'étudiant s'inscrit à l'événement (statut : EN_ATTENTE).
2. Il effectue le paiement via une application tierce (Bankily, Masrivi) et envoie la preuve au responsable.

3. Le responsable valide le paiement dans l'onglet Trésorerie de son panel.
4. Le statut passe à PAYÉ et une commission (10%) est automatiquement enregistrée dans la table commissions.
5. L'administrateur facture l'association mensuellement sur la base de la table commissions.

Quatrième partie : IV Méthodologie et Qualité

8 Méthodologie Scrum Adaptée et Suivi des Sprints

Le projet CampusConnect a été développé selon le framework Scrum, adapté à notre contexte spécifique (équipe réduite et contraintes académiques).

8.1 Adaptations de la Méthodologie Scrum

Organisation Scrum adaptée :

- **Moussa Mohamed Erebi** : Scrum Master + Développeur
- **Saleck Ahmed Baya** : Product Owner + Développeur
- **Daily Scrum** : Tous les 2 jours au lieu de quotidien (contraintes de planning)
- **Sprints** : Durée de 1 semaine (cycles courts pour itérations rapides)
- **User Stories** : Environ 6 US par sprint, répartition équitable entre les deux développeurs

8.2 Suivi Détaillé des Sprints

8.2.1 Sprint 0 : Initialisation et Cadrage

Durée : 1 semaine

Objectif : Préparer l'environnement de travail et définir la vision du produit.

TABLE 8.1 – Sprint 0 — US réalisées

US	Description	Responsable	Statut
US01	Vision Produit et Personas	Moussa	Terminé
US02	Backlog du produit (première version)	Saleck	Terminé
US03	Definition of Done (DoD)	Moussa	Terminé

US	Description	Responsable	Statut
US04	Règles pour bosser ensemble	Saleck	Terminé
US05	Mise en place de l'environnement	Moussa	Terminé*
US06	Validation de l'architecture MVC	Saleck	Terminé

Difficulté rencontrée : Problème de versions Node.js sur les deux postes (US05).

Solution (Scrum Master) : Mise en place d'un fichier `.nvmrc` pour fixer la version et garantir l'uniformité de l'environnement.

8.2.2 Sprint 1 : Authentification et Gestion des Associations

Durée : 1 semaine

Objectif : Mettre en place le système d'authentification et le workflow de création d'associations.

TABLE 8.2 – Sprint 1 — US réalisées

US	Description	Responsable	Statut
US07	Créer un compte (invité)	Moussa	Terminé
US08	Envoyer un email de confirmation	Saleck	Terminé*
US09	Se connecter et changer mot de passe	Moussa	Terminé
US10	Demander à devenir responsable	Saleck	Terminé*
US11	L'admin valide la demande	Saleck	Terminé
US12	Gérer le profil de l'association	Moussa	Terminé
US13	Voir l'annuaire des associations	Saleck	Terminé

Difficultés rencontrées : - Emails (US08) : Gmail bloquait la connexion SMTP.

Solution : Activation de la double authentification et création d'un mot de passe d'application. - Upload de logo (US10) : Problème de chemins sur Windows. Solution : Utilisation de `path.join` et création automatique du dossier `uploads/`.

8.2.3 Sprint 2 : Adhésions et Événements

Durée : 1 semaine

Objectif : Développer le cœur métier : adhésions aux associations et gestion d'événements.

TABLE 8.3 – Sprint 2 — US réalisées

US	Description	Responsable	Statut
US14	Demander à adhérer	Moussa	Terminé
US15	Accepter ou refuser une adhésion	Saleck	Terminé
US16	Accès à l'espace membre	Moussa	Terminé
US17	Publier un événement	Saleck	Terminé
US18	Vérifier si une salle est déjà prise	Moussa	Terminé*
US19	Voir tous les événements	Saleck	Terminé
US20	Appliquer les tarifs automatiquement	Moussa	Terminé
US21	S'inscrire / se désinscrire	Saleck	Terminé
US22	Voir la liste des inscrits	Saleck	Terminé

Difficulté rencontrée : L'US18 (algorithme anti-conflit) a nécessité une requête SQL complexe sur la détection de chevauchements d'intervalles. **Solution :** Pair programming entre le SM et le PO pour valider la condition ($StartA < EndB \text{ AND } EndA > StartB$).

8.2.4 Sprint 3 : Notifications, Paiements et Tableaux de Bord

Durée : 1 semaine

Objectif : Mettre en place le système de notifications, la gestion des preuves de paiement et les dashboards.

TABLE 8.4 – Sprint 3 — US réalisées

US	Description	Responsable	Statut
US23	Email de confirmation d'inscription	Moussa	Terminé
US24	Rappel automatique 24-48h avant	Saleck	Terminé
US25	Notifier si un événement change	Moussa	Terminé
US26	Upload d'une preuve de paiement	Saleck	Terminé
US27	Valider manuellement un paiement	Moussa	Terminé
US28	Dashboard pour les responsables	Saleck	Terminé
US29	Statistiques pour l'admin	Saleck	Terminé
US30	Historique de l'utilisateur	Moussa	Terminé

Amélioration proposée par le PO : Ajout d'un lien WhatsApp pour faciliter la communication entre étudiants et responsables (fonctionnalité non prévue initialement). **Décision SM :** Acceptée et intégrée directement dans le frontend.

8.2.5 Sprint 4 : Freemium, Sécurité et Livraison

Durée : 1 semaine

Objectif : Finaliser le modèle Freemium, renforcer la sécurité et préparer la livraison.

TABLE 8.5 – Sprint 4 — US réalisées

US	Description	Responsable	Statut
US31	Restrictions du plan Freemium	Moussa	Terminé
US32	Calcul automatique des commissions	Saleck	Terminé
US33	Contrôle d'accès (RBAC)	Moussa	Terminé
US34	Chiffrer les données sensibles	Saleck	Terminé*
US35	Interface responsive	Moussa	Terminé
US36	Tests d'intégration	Moussa	Terminé
US37	Optimiser les requêtes SQL	Saleck	Terminé
US38	Démo et documentation	Moussa	Terminé

Amélioration sécurité : L'US34 a été enrichie avec un module `encryption.js` utilisant AES-256-CBC pour chiffrer les numéros de téléphone (au lieu du simple hachage initialement prévu).

8.3 Synthèse des Sprints

TABLE 8.6 – Suivi global des Sprints Scrum

Sprint	Points	Statut	Livrables principaux
Sprint 0 — Initialisation	22	Terminé	Vision produit, personas, backlog initial, DoD, environnement Git, architecture MVC validée
Sprint 1 — Auth & Associations	23	Terminé	Inscription, confirmation email, JWT, reset MDP, validation admin, annuaire associations
Sprint 2 — Adhésions & Événements	31	Terminé	Adhésion, anti-conflit salle, catalogue événements, inscriptions, tarification
Sprint 3 — Notifications &	29	Terminé	Emails auto (Nodemailer), cron rappels J-1/J-2, upload preuve paiement, trésorerie

Sprint	Points	Statut	Livrables principaux
Dashboards			
Sprint 4 — Freemium & Livraison	29	Terminé	RBAC complet, commissions 10%+5%, sponsors, révocation Premium auto, Swagger, tests
Total	134	100%	38 User Stories complétées

8.3.1 Leçons apprises et bonnes pratiques

Retour d'expérience Scrum :

- Les sprints d'1 semaine ont permis une itération rapide et une détection précoce des problèmes.
- Le pair programming a été efficace pour résoudre les blocages techniques complexes (anti-conflit, chiffrement).
- L'adaptation des daily meetings à une fréquence de 2 jours a suffi pour une équipe de 2 personnes.
- Le Product Owner a pu ajouter des fonctionnalités à valeur ajoutée (WhatsApp, chiffrement avancé) en cours de sprint grâce à la flexibilité agile.

9 Sécurité et Qualité du Code

9.1 Authentification et Autorisation

- JWT stateless avec expiration 24h, signé avec une clé secrète côté serveur.
- Mots de passe hachés avec bcrypt (Round 10) — irréversible.
- Middleware requireRole : vérifie le rôle décodé du token avant d'accéder à chaque route protégée.
- Distinctions claires : compte Non Activé (email non confirmé) vs Suspendu (modération admin).
- Token stocké en localStorage, injecté automatiquement via un intercepteur Axios global.

9.2 Sécurité du Serveur

- **Helmet.js** : headers de sécurité HTTP (XSS, clickjacking, MIME sniffing).
- **CORS** configuré pour autoriser uniquement le frontend (localhost:5173).
- Validation des uploads avec Multer (type MIME, taille max).
- Requêtes SQL préparées (paramétrisées) via mysql2 pour prévenir les injections SQL.

- `crossOriginResourcePolicy` configuré pour les images dynamiques (/uploads/).
- Chiffrement AES-256-CBC pour les données sensibles (numéros de téléphone).

10 Tests et Audit

10.1 Tests Fonctionnels Validés

- **Anti-conflit de salle** : chevauchements partiels et totaux testés — HTTP 409 retourné correctement.
- **Workflow de paiement MVP** : EN_ATTENTE → PAYÉ → commission 10% enregistrée en base.
- **Cron jobs** : tâche 0 8 * * * validée —> scan J-1/J-2 et envoi d'emails de rappel.
- **RBAC** : toutes les routes protégées retournent HTTP 403 si le rôle est insuffisant.
- **Freemium** : révocation automatique quotidienne des associations en plan expiré.
- **Tarification** : remise 20% correctement appliquée aux 10 premiers inscrits.

10.2 Points d'Attention Identifiés

Ces points sont documentés pour les futures itérations :

- Validation des types MIME côté frontend pour les uploads (renforcement recommandé).
- Ajout d'index SQL sur les colonnes fréquemment filtrées (`event_id`, `user_id`, `status`).
- Migration vers des tests automatisés Jest/Supertest pour les routes API critiques.
- Gestion des tokens expirés : redirection automatique vers la page de connexion.

Cinquième partie : V Bilan et Conclusion

11 Bilan et Perspectives

11.1 Ce qui a été livré

Livrables principaux du projet :

- Plateforme Full-stack fonctionnelle (React 18 + Node.js + MySQL).
- 5 sprints Scrum complétés : 134 points story livrés (100% du backlog).
- 38 User Stories réalisées avec succès.
- Système RBAC complet à 4 niveaux avec JWT et bcrypt.
- Commissions automatisées : 10% sur billetterie, 5% sur cotisations.
- 5 sponsors corporatifs réels intégrés avec logos et espace dynamique.
- Documentation Swagger complète et interactive sur /api-docs.
- Cron jobs opérationnels : rappels, révocation Premium, archivage.
- Module de dons pour les ONG avec gestion de commission.
- Chiffrement des données sensibles (AES-256-CBC).

11.2 Perspectives v2.0

- Intégration d'une passerelle de paiement en ligne (Stripe ou API Bankily).
- Développement d'une application mobile React Native.
- Messagerie interne entre membres et responsables d'association.
- Tableau de bord analytique avancé (D3.js, graphiques interactifs).
- Déploiement cloud sur un VPS ou hébergeur mauritanien.
- Système de votes et sondages associatifs.
- Module de candidature aux postes de bureau d'association.

Conclusion

CampusConnect répond de manière concrète et opérationnelle au défi de la numérisation de la vie associative universitaire. En combinant une architecture technique solide, un modèle économique viable et une expérience utilisateur soignée, la plateforme constitue une base robuste pour une mise en production réelle.

Le projet démontre la maîtrise d'un cycle de développement Fullstack complet, de l'analyse des besoins à la livraison, en passant par la conception, l'implémentation et les tests, le tout dans un cadre agile Scrum rigoureusement adapté à notre contexte. L'adaptation de la méthodologie (sprints d'1 semaine, daily bi-journaliers) a permis une livraison itérative de valeur tout en respectant les contraintes académiques.

CampusConnect est prêt pour une phase pilote à la FST ou toute autre type d'association (ONG, CLUB...).